

Master's Thesis

Adaptive Rounding for Efficient Post-Training Quantization of Deep Neural Networks

48246347 Piye Liu

Supervisor Prof. Hiroshi Nakamura

August, 2026

Department of Information Physics and Computing
Graduate School of Information Science and Technology
The University of Tokyo

概要

Abstract in Japanese should be written here.

Abstract

300 words abstract in English should be written here.

Contents

Chapter 1	Introduction	1
Chapter 2	Preliminary	2
2.1	Compressibility of Deep Neural Networks	2
2.2	Baseline Round-to-Nearest Quantization	2
2.3	Quantization is Weight Perturbation	2
2.4	Small Language Models	3
2.5	Effects of Perturbation of Model Weights	4
Chapter 3	Literature Review	5
Chapter 4	Proposed Method	6
4.1	AdaRound with Gradient Awareness	6
Chapter 5	Evaluation	8
Chapter 6	Conclusion	9
	Bibliography	10
A	Appendix	11

Chapter 1

Introduction

Chapter 2

Preliminary

Source coding theorem
+ MLE estimation of
Qwen3's entropy here

2.1 Compressibility of Deep Neural Networks

2.2 Baseline Round-to-Nearest Quantization

The baseline for post-training quantization is symmetric uniform quantization using round-to-nearest (RTN) rounding. [1]

Algorithm 1 Baseline RTN Quantization

Require: W (unquantized weights), b (target bit width)

Ensure: Quantized model W_q

- 1: $q_{\min} \leftarrow -2^{b-1}$
 - 2: $q_{\max} \leftarrow 2^{b-1} - 1$
 - 3: $\text{scale} \leftarrow \frac{\max(W)}{q_{\max}}$
 - 4: $W_q \leftarrow \text{clamp}(\lceil \frac{W}{\text{scale}} \rceil, q_{\min}, q_{\max})$
 - 5: **return** W_q, scale
-

Algorithm 2 Baseline RTN Dequantization

Require: W_q (quantized weights), scale

Ensure: Dequantized weights $\hat{W}$

- 1: $\hat{W} \leftarrow W_q \cdot \text{scale}$
 - 2: **return** $\hat{W}$
-

2.3 Quantization is Weight Perturbation

Let us establish the observation that quantization results in a perturbed model, using an example. Consider a model θ with parameters $W = \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix}$. By applying the baseline round-to-nearest quantization from Section 2.2, we obtain the following quantized model.

$$\text{scale} \approx 0.0262, W_q = \begin{bmatrix} 0\text{b}00101010 \\ 0\text{b}01010101 \\ 0\text{b}01111111 \end{bmatrix} = \begin{bmatrix} 42 \\ 85 \\ 127 \end{bmatrix}$$

After dequantization, we obtain the following model.

$$W_{\text{reconstructed}} = \text{scale} \cdot W_{\text{quantized}} \approx \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix}$$

We can see that the model is perturbed by the following.

$$\Delta W = \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix} - \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix} = \begin{bmatrix} -0.0087 \\ 0.0087 \\ 0 \end{bmatrix}$$

It is important to note the direction of perturbation. In this case, our model moved in the direction of $\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$.

2.4 Small Language Models

In this section, we will examine TinyStories [2], a work that pushes the boundaries of small language models, and provide a few custom language models for further experimentation. By using a low-variance synthetic dataset (children’s stories) generated by GPT-3.5 and GPT-4 containing only words a typical 3-4 year old would understand, they were able to generate coherent output using models with as few as 1 million parameters.

One limitation we observed in this work is their usage of the GPT-Neo [3] tokenizer, restricted to the top 10,000 tokens. Using a generic tokenizer for a low-vocabulary dataset like TinyStories might result in wasted weights (tokens that never appear in TinyStories), and might not provide the optimal tokenization for our specialized dataset.

As a preliminary experiment, we train two small language models on TinyStories v1 and TinyStories v2, respectively. We use a SentencePiece Byte-Pair Encoding (BPE) tokenizer [4] with a vocabulary size of 1024, and we used a LLaMa-based [5] model with roughly half the parameters of the smallest reference model.

2.4.1 Evaluation: Quantitative Metric

We use perplexity [?] as a quantitative metric to evaluate a language model’s accuracy. For a model θ , a text x of length N , it is defined as the following.

$$\text{Perplexity}(\theta, x) = \left(\prod_{i=1}^N \frac{1}{p_{\theta}(x_i | x_1, x_2, \dots, x_{i-1})} \right)^{-\frac{1}{N}} \quad (2.1)$$

In other words, it is the geometric mean of the inverse perplexities of a given text. To show this with an example, suppose we have a model θ and a ground truth text “Mary had a little lamb”. Suppose that the model’s predicted probabilities of this text is the following.

Table 2.1: Example Model Output Probabilities

Item	Value
$p_{\theta}(\text{“Mary”})$	0.5
$p_{\theta}(\text{“had”} \text{“Mary”})$	0.7
$p_{\theta}(\text{“a”} \text{“Mary”, “had”})$	0.9
$p_{\theta}(\text{“little”} \text{“Mary”, “had”, “a”})$	0.6
$p_{\theta}(\text{“lamb”} \text{“Mary”, “had”, “a”, “little”})$	0.9

Then the perplexity of the model on the text is the following.

$$\begin{aligned} \text{Perplexity} &= \left(\frac{1}{0.5} \cdot \frac{1}{0.7} \cdot \frac{1}{0.9} \cdot \frac{1}{0.6} \cdot \frac{1}{0.9} \right)^{\frac{1}{5}} \\ &\approx 1.49 \end{aligned}$$

Intuitively, perplexity is a measure of how surprised or confused a model is when seeing a certain text. We compute the model’s perplexity over its validation data, then use the result as a metric to evaluate the model’s accuracy (lower is better).

2.4.2 Evaluation: Results and Discussion

We report the validation perplexity of our two models. Note that the original paper only provided models trained on TinyStories v1.

Table 2.2: Evaluation of Our Language Models

	Reference	Ours (TinyStories v1)	Ours (TinyStories v2)
Architecture	GPT-Neo	LLaMa	LLaMa
Parameter Count	1.17M	0.57M	0.46M
Training Data	TinyStories v1		TinyStories v2
Tokenizer (Vocab Size)	GPT-Neo	SentencePiece	SentencePiece
Vocab Size	10000	1024	1024
Perplexity (TinyStories v1)	19.3919	11.9006	—
Perplexity (TinyStories v2)	—	—	8.0994

Add qualitative results (actual model outputs) here

We observe that our model results in better perplexity than the reference model despite being much smaller. In `float32`, each weight is 4 bytes, so our model is expected to take only $\sim 2.17\text{MiB}$ of disk space (the actual disk space, including the model’s configuration, is $\sim 2.22\text{MiB}$). Models of this scale can be suitable for devices with extremely low compute and/or memory.

We will use our models as a base to conduct several more experiments.

2.5 Effects of Perturbation of Model Weights

As we established in Section 2.3, quantization perturbs weights in a certain direction. In this section, we examine the effects of weight perturbation using our TinyStories v2 model, by viewing its loss landscape.

2.5.1 Language Model Optimization Process

To understand our model’s loss landscape, we must first understand its loss function.

Cross entropy loss explainer here

2.5.2 Loss Landscape

2.5.3 Discussion

Chapter 3

Literature Review

Chapter 4

Proposed Method

4.1 AdaRound with Gradient Awareness

To reconcile AdaRound’s assumption that the gradient term in ?? can be ignored, and Central Flows’ evidence that the gradient is nonzero even for pretrained converged models, we propose a modified version of AdaRound to incorporate awareness of the local gradient.

In order to minimize,

$$\mathbb{E} \left[g^{(w)^T} \Delta W \right], \quad (4.1)$$

we can attempt to force ΔW to have the same sign as $-g^{(w)^T}$. We approximate this by finetuning a model on a small amount of training data, and compute the Exponential Moving Average (EMA) of the model during each iteration. We then use $\text{sign}(W'_{\text{avg}} - W)$, the sign of the difference between the weights before and after finetuning, as a proxy to $-\text{sign}(g^{(w)^T})$. Finally, we add the Binary Cross Entropy loss between the current rounding directions and $\text{sign}(W'_{\text{avg}} - W)$ as a penalty term to AdaRound’s optimization objective.

Algorithm 3 Proposed Method (Finetuning)

Require: W , X (calibration data), η (learning rate), B (batch size)

Ensure: Finetuned model W' , EMA of models during finetuning W'_{avg}

```

1:  $W' \leftarrow W$ 
2:  $W'_{\text{avg}} \leftarrow W$ 
3:  $N \leftarrow \frac{X}{B}$  ▷ Number of Iterations
4:  $\alpha \leftarrow \frac{2}{N+1}$  ▷ EMA Smoothing Factor
5: for each batch  $x_b$  in  $X$  do
6:    $\hat{x}_b \leftarrow W'(x_b)$  ▷ Forward pass
7:    $\mathcal{L} \leftarrow \text{CrossEntropyLoss}(\hat{x}_b, x_b)$ 
8:    $W' \leftarrow W' - \eta \cdot \nabla \mathcal{L}$  ▷ Backward pass
9:    $W'_{\text{avg}} \leftarrow \alpha \cdot W'_{\text{avg}} + (1 - \alpha) \cdot W'$ 
10: end for
11: return  $W'$ ,  $W'_{\text{avg}}$ 

```

Algorithm 4 Proposed Method (Calibration)

Require: W, W'_{avg}, X (calibration data), η (learning rate), t (number of iterations)**Ensure:** Quantized model W_q

1: (TODO)

2: **return** W_q

Chapter 5

Evaluation

Chapter 6

Conclusion

Bibliography

- [1] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A White Paper on Neural Network Quantization, June 2021.
- [2] Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English?, May 2023.
- [3] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large scale autoregressive language modeling with meshtensorflow. Zenodo, October 2021.
- [4] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing, August 2018.
- [5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models, February 2023.

A

Appendix